ENPIRE: Agentic Robot Policy Self-Improvement in the Real World

Wenli Xiao[1,2*], **Jia Xie**[2*], **Tonghe Zhang**[2*], **Haotian Lin**[2*], **Letian "Max" Fu**[3], **Haoru Xue**[3], **Jalen Lu**[2]
Yi Yang[2], **Cunxi Dai**[2], **Zi Wang**[1], **Jimmy Wu**[1], **Guanzhi Wang**[1], **S. Shankar Sastry**[3], **Ken Goldberg**[3]
Linxi "Jim" Fan[1,†], **Yuke Zhu**[1,†], **Guanya Shi**[2,†]
[1]NVIDIA [2]CMU [3]UC Berkeley [*]Equal contribution [†]Equal advising

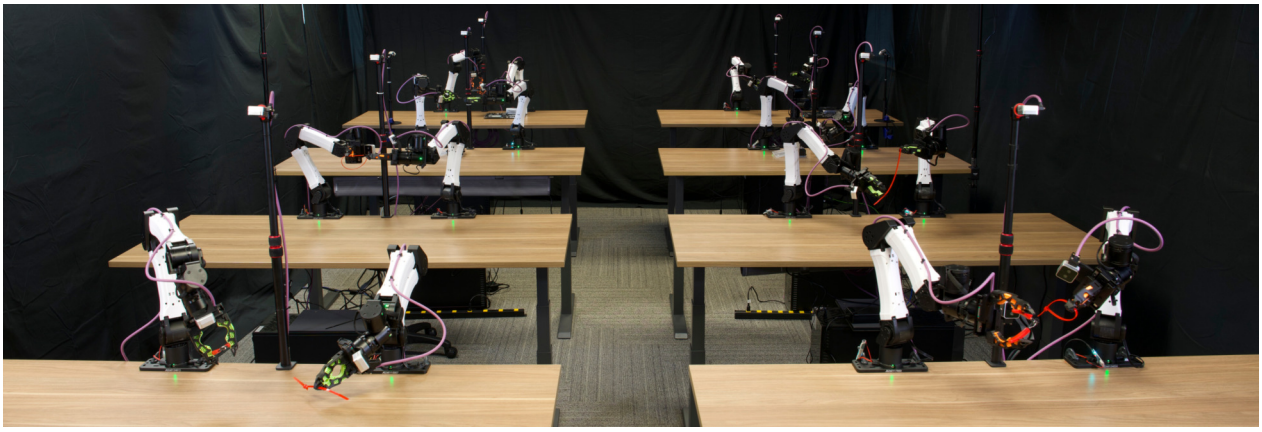


Figure 1: **Robot fleet for physical autoresearch**. The fleet contains eight bimanual YAM robot stations. Each station owns its robot hardware, compute, and coding agent. Website: research.nvidia.com/labs/gear/enpire

Abstract

Achieving dexterous robotic manipulation in the real world relies heavily on human supervision and algorithmic engineering, which is a central bottleneck in the pursuit of general physical intelligence. Although emerging coding agents can generate code to automate algorithm search, their successes remain largely confined to digital environments. We conjecture that the missing abstraction to automate robotics research is a *repeatable feedback loop* for real-world policy improvement: reset the scene, execute a policy, verify the outcome, and refine the next iteration. To bridge this gap, we introduce ENPIRE, a harness framework for coding agents that instantiates this physical feedback routine with four core modules: an Environment module (EN) for automatic reset and verification, a Policy Improvement module (PI) that launches policy refinement, a Rollout module (R) to evaluate policies with single or multiple physical robots operating in parallel, and an Evolution module (E) in which coding agents analyze logs, consult literature, improve training infrastructure and algorithm code to address failure modes. This closed-loop system transforms real-world robot learning into a controllable optimization procedure that agents can manage, thus minimizing human effort while allowing fair ablations across training recipes and agent variants. Powered by ENPIRE, frontier coding agents can autonomously develop a policy to achieve a 99% success rate on challenging, dexterous manipulation tasks in the real world, such as PushT, organizing pins into a pin box, and using a cutter to cut a zip tie. Coding agents can improve policies with various PI regimes, such as heuristic learning, tool calling, behavior cloning, offline or online reinforcement learning. Moreover, ENPIRE can be significantly accelerated on a robot fleet, and we propose two metrics, namely, Mean Robot Utilization (MRU) and Mean Token Utilization (MTU) to measure the efficiency of multi-agent physical autoresearch. We also include simulation results in RoboCasa. Our findings suggest a practical and scalable path towards autonomously advancing robotics in the real world.

© 2026 NVIDIA. All rights reserved.

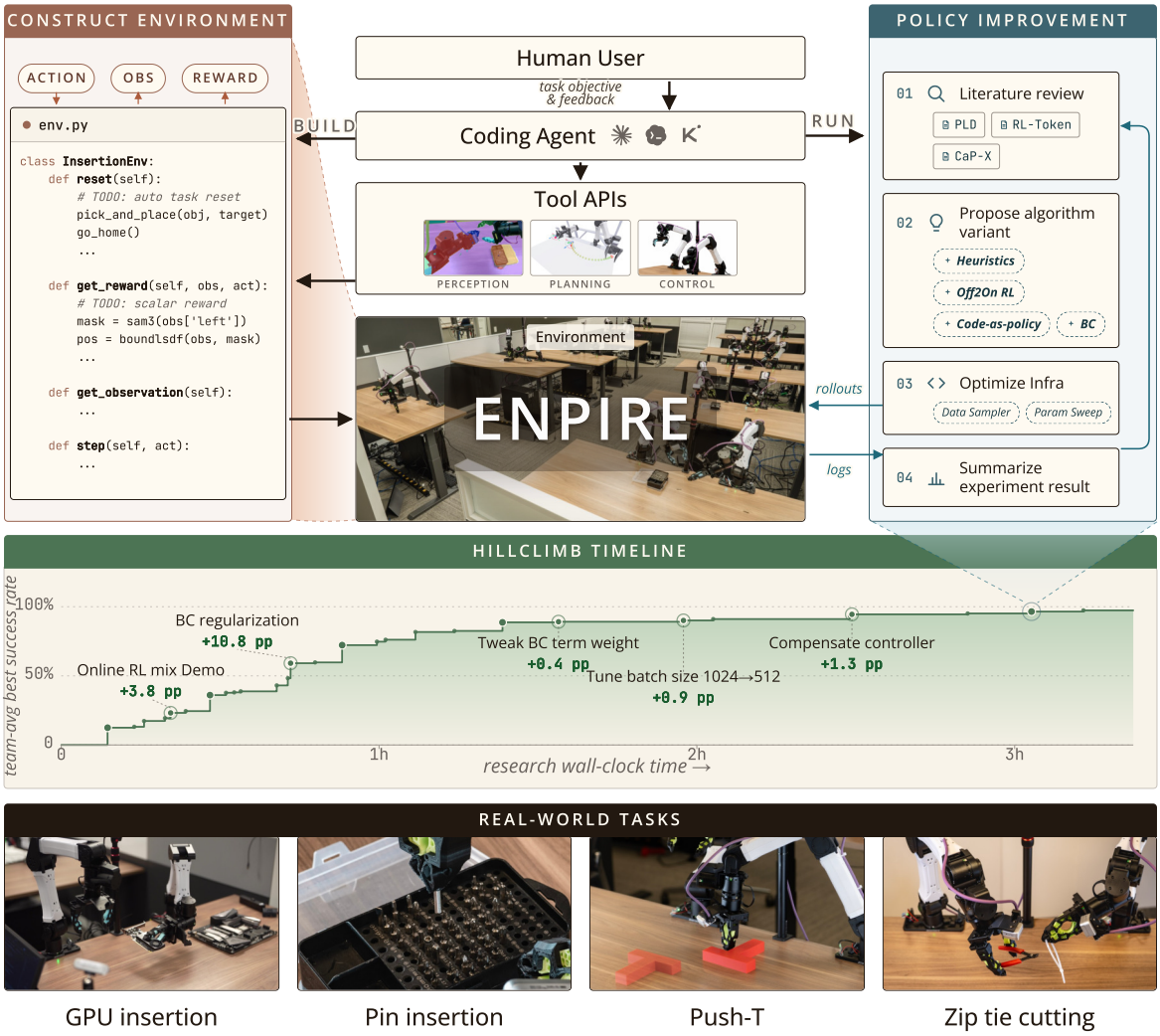


Figure 2: Overview of physical autoresearch framework **ENPIRE**. To solve a dexterous task in the real world, **ENPIRE** first uses tool calls to construct an environment with automatic reset and verification mechanisms according to human feedback. Next, the coding agent calls environment APIs to conduct autoresearch and hill-climb the policy success rate from real-world reward signals. More results are available at research.nvidia.com/labs/gear/enpire.

1. Introduction

Automatically learning dexterous manipulation skills has been a major obstacle in the journey towards general physical intelligence. Frontier policy training methods, although highly effective, still rely on humans behind the scenes to participate in the data collection, evaluation with reset, and algorithm adjustment life cycle (19; 20; 24). As we scale up real-world policy learning, humans labor of babysitting policy improvement inevitably become one of the limiting factors of how quickly robots acquire dexterity.

Recent advances in autonomous research (22) (autoresearch), powered by coding agents, show a promising path toward automating algorithmic improvements. However, unique challenges arise when we extend this paradigm to automate real-world policy learning. First, coding agents lack a set of *real-world environment interfaces* for closed-loop hypothesis testing in the physical world, which requires automated policy deployment, evaluation, and scene resetting. Second, when *scaling up* autoresearch throughput across robot fleet, it remains an open problem to select and verify hypotheses while maintaining efficiency of resource utilization.

To address these challenges, we introduce **ENPIRE**, an agent harness framework that leverages a suite of autonomous environment interfaces to enable scalable physical autoresearch. **ENPIRE** decomposes the acquisition of dexterous manipulation skills into two autoresearch procedures. In the first phase (**EN** in **ENPIRE**), coding agents construct an autonomous environment interface from human feedback. Through this initial research loop, agents implement and optimize procedural tool calls that establish safety boundaries, automated reset, and verification procedures for a specific task. They are optimized offline with a one-time setup cost; once finalized, they serve as immutable APIs that are reused throughout the subsequent stage. The second phase (**PIRE** in **ENPIRE**) transitions to a fully autonomous autoresearch procedure, where coding agents refine policies based on real-world feedback. Guided entirely by the environment's automated verification signals, the agents independently explore and optimize diverse methodologies to maximize real-world task success rates without human intervention.

To scale this pipeline across multiple robots in parallel, **ENPIRE** introduces a mechanism to evolutionarily select hypotheses, which accelerates policy improvement. We dispatch a decentralized team of agents to test training recipes asynchronously, sharing and abandoning ideas based on the average success rates. The accumulated knowledge can be transferred to similar, novel tasks.

We highlight the contributions of this work as follows:

- We formalize physical autoresearch for dexterous manipulation as a distinct problem for the coding agent and robotics communities. We propose to tackle this with a two-step approach: first, conducting a human-guided autoresearch to construct automatic environment feedback and verify it offline, then executing a fully autonomous autoresearch from real-world feedback in an online fashion to improve the policy.
- We implement **ENPIRE**, an agentic harness through which a coding agent develops reusable robotic tools, constructs rewards and reset mechanisms with procedural tool calls, and effectively optimizes learning algorithms on real robots without human intervention.
- We demonstrate that **ENPIRE** can hill-climb the success rate to a high level in multiple dexterous manipulation tasks. In pin insertion, policy convergence to 100% is achieved faster than a frontier human-in-the-loop method (48). We also observe initial scaling behavior for parallelized autoresearch on a robot fleet and propose key metrics, Mean Token Utilization (MTU) and Mean Robot Utilization (MRU), to benchmark the efficiency of this procedure.

2. Method

In this section, we present a harness design that allows coding agents to perform automated research in the physical world. This procedure is decomposed into two stages: *environment construction from human feedback* (**EN** in **ENPIRE**) and *automatic policy improvement from real-world feedback* (**PIRE** in **ENPIRE**).

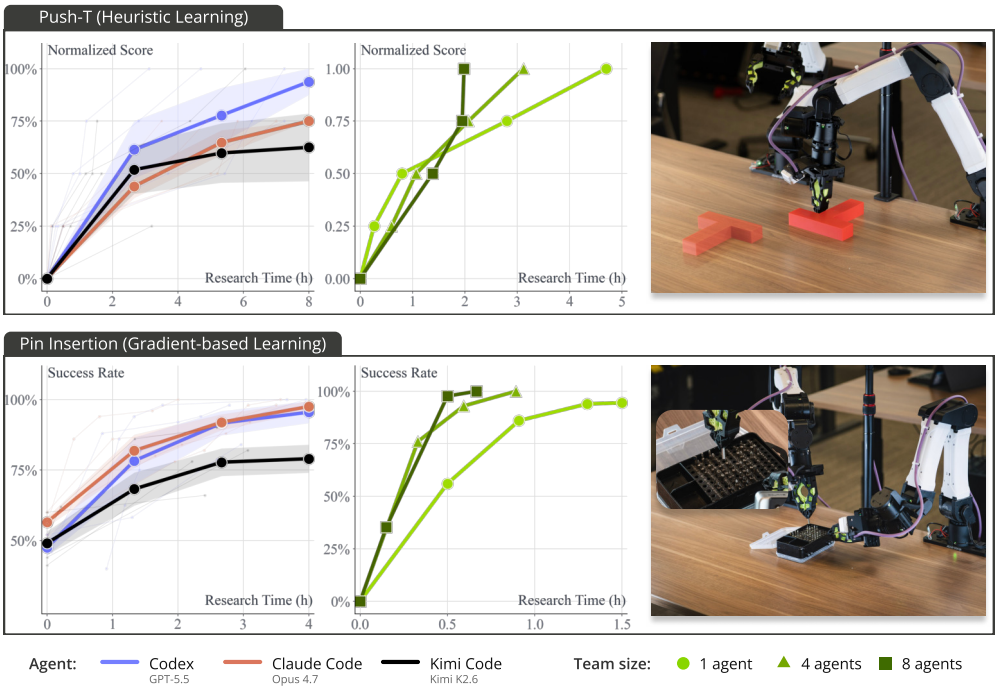


Figure 3: **Benchmarking coding agents for physical autoresearch**. **ENPIRE** enables `state-of-the-art` coding agents to achieve autonomous policy improvement on **Push-T** and **Pin Insertion** tasks. **ENPIRE** also scales with resources, as increasing the number of robot agent workers reduces the wall-clock time required to reach the same task performance.

2.1. Stage One: Environment Construction from Human Feedback

For coding agents to conduct autonomous physical autoresearch, we need to first construct an agent-friendly abstraction of physical interaction and feedback as a structured environment. This includes task-specific safety constraints to support long-term operation, a real-time automated verification process for feedback and credit assignment, and a robust automatic reset mechanism to ensure fast iteration. To improve reliability, agents construct the environment APIs with procedural tool calls and refine their implementation according to human assessment. Human effort is a one-time cost that will be amortized across all implementations on all robots during subsequent automatic policy improvement in Sec. 2.2.

Hard safety constraints

We restrict the configuration space and kinematic behaviors of the robot to a safe operational limit. The safety regions are sufficient for task completion and serve as hard constraints: violating these limits triggers an immediate task failure and an automated reset. This serves as a safeguard for real-world interaction, also as a source of episode termination or truncation.

Automated verification

During real-world experiments, robot learning needs real-time verification from sensor inputs to quantify experimental progress. To minimize human engineering efforts, coding agents are tasked with synthesizing a binary reward function from procedural tool calls to distinguish task outcomes. Given only a few minutes of success and failure demonstrations, the agents use the videos and proprioception recordings to maximize the prediction accuracy score while minimizing processing latency. For example, autoresearch discovers a robust reward function for pin insertion that is based on visual alignment, end-effector height, and force estimates. Coding agents also show their ability to design perception-dependent rewards in zip-tie insertion in Fig. 4

and optimize its inference latency to under 150ms, which approaches the reactiveness of the human visual system (43). More details are provided in the appendix.

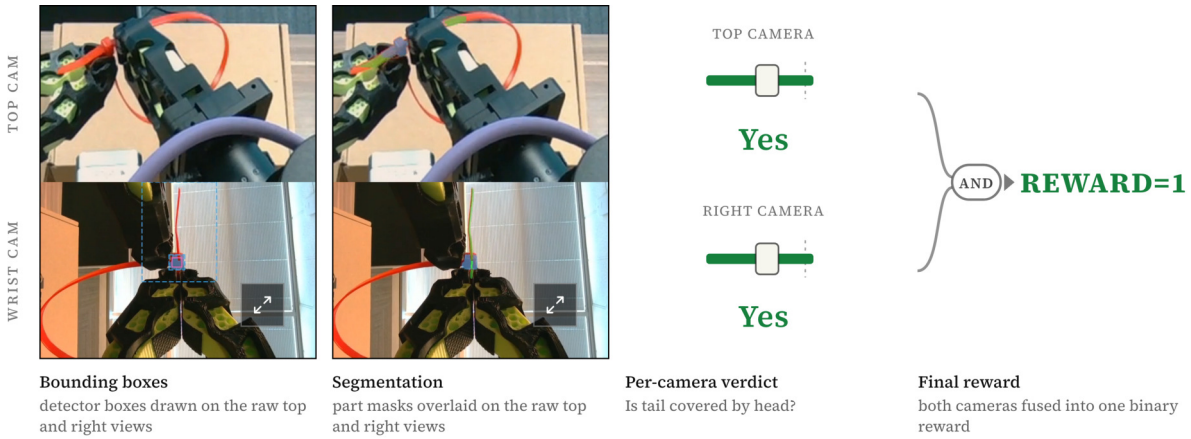


Figure 4: Reward for zip-tie insertion. Cropping and image segmentation test whether the zip-tie strap passes through the zip-tie head. Two camera views are considered to prevent false positives.

Automated reset

Once a task is completed or fails, the agent executes a series of tool calls to restore the environment to its initial state. For contact-rich tasks, we employ procedural tool calls inspired by CaP-X (15), using modular manipulation skills to reset the environment directly to the start of the most challenging phase. By positioning the robot at the precise onset of critical actions, such as inserting a pin, seating a GPU, or grasping scissors, we strategically focus the learning system on these precision bottlenecks.

The safety constraints, automated verification, and automated reset constitute our Environment module (**EN** in **ENPIRE**). Policies also receive visual-proprioception sensor inputs and submit action commands to the robot controller, which, combined with our reward, constitutes the Rollout module (**R** in **ENPIRE**). Once constructed, coding agents have access to these modules through immutable Gym APIs (7), which provide feedback signals and debugging information for automated policy improvement.

2.2. Stage Two: Automated Policy Improvement from Real-World Feedback

The second stage leverages automated research to train a policy for a specific task. Upon initialization, the coding agent receives the task description with the ultimate objective of maximizing the task's success rate through autonomous experimentation. To achieve this, the agent is granted write permissions to a streamlined training codebase that supports basic end-to-end policy training and code-based policy synthesis.

A motivating example

We illustrate this process using a pin insertion task, where the policy must insert a pin into a hole with a tight 4mm clearance (Fig. 2). During this automated research phase, the agent operates through a Policy Improvement module (**PI** in **ENPIRE**), where it reviews the literature to generate insights, formulates hypotheses, and directly modifies the training code—such as behavior cloning or RL algorithms—to optimize performance based on real-world automatic verification results. To gather rich evidence, the agent invokes environment APIs to log robot trajectories, video recordings, and reward signals during rollouts, and inspects these statistics to guide continued improvement.

Accelerating autoresearch via multi-agent scaling

While automatic reset and verification mechanisms enhance the scalability of a single deployment loop, physical autoresearch can be further accelerated by launching a parallel, multi-agent decentralized collaboration

protocol. This protocol deploys N agents across N physical robots to test N hypotheses asynchronously. Each agent branches off from the same baseline policy training codebase, collaborating autonomously via Git to scale this Evolution module (**E** in **PIRE**). Without human intervention, agents spontaneously cherry-pick, copy, or merge successful training recipes from their peers to optimize their code search. Empirically, we observe that scaling the number of parallel agent-robot pairs drastically reduces the wall-clock time required to discover a high-success-rate policy recipe, as illustrated in Fig. 3.

To quantify how efficiently a coding agent converts its allocated physical resources into research progress, we propose two utilization metrics that complement task-level outcomes. Mean Robot Utilization (MRU) is the fraction of research wall-clock time during which the robot is actively executing an experiment. GPU utilization is the fraction of research wall-clock time during which the GPU is actively in use. A perfectly resource-saturated agent would push both metrics toward 1; in practice, neither metric reaches this value for any frontier coding agent we evaluate. To measure how efficiently an agent team converts tokens into successful robot policies, we define Mean Token Utilization (MTU) as the fleet's average token consumption throughout autoresearch. We then calculate the token-to-success ratio to reflect the token efficiency of task learning. An empirical result of how robot and token utilization changes with agent team size is illustrated in Fig. 7.

3. Experiment

We show that **ENPIRE** is a capable and scalable agentic autoresearch framework.

- *First, **ENPIRE** supports the autonomous policy learning after environment construction.*
- *Second, the convergence speed of the success rate **ENPIRE** scales with robot and token resources.*

We will focus on the following dexterous tasks, which require precise and reactive control from perceptual feedback: **Push-T** (12), where the robot uses non-prehensile movement to align a T-shape block to a goal region; **Pin insertion**, where the robot organizes a pin box by plugging in pins into 4mm-diameter holes; **GPU-insertion**, where the robot seats GPU chips in thin sockets on the motherboard, and **Ziptie-cutting**, where the robot grabs and closes scissors to cut the tail of a zip tie. A visualization of these tasks is provided in Fig. 2.

We measure success as the chance of completing a task in one rollout, given a fixed number of retries (here, eight). Unlike measuring success by i.i.d. best-of-N sampling, where each attempt is independent, our retries happen after witnessing the previous failure, implying that our metric captures both precision and in-context recovery in the face of environment uncertainty. We stress that recovery, not just one-shot precision, matters for highly dexterous tasks and reflects the robustness needed for reliable deployment in the real world.

We use **ENPIRE** to conduct autoresearch in various policy learning paradigms. Coding agents have the freedom to choose various methods and their combinations to solve a task, including end-to-end neural network training such as behavior cloning (BC) (36) or real-world reinforcement learning (RL) (18), as well as gradient-free learning methods such as heuristic learning (47) and code-based policy synthesis (25). Our real robot platform is the bimanual 6-DoF YAM robot. We benchmark three coding agents in our physical autoresearch experiments, namely, Codex with GPT-5.5 xhigh (35), Claude Code with Opus 4.7 High (3), and Kimi Code with Kimi K2.6 thinking (33).

3.1. Autoresearch for Heuristic Learning

We tested **ENPIRE**'s ability to drive automatic policy improvement in its simplest form: learning heuristics (47) by synthesizing perception and control tool calls (25). To this end, we built a real-world **Push-T** environment with its simulation counterpart for comparison.

The unique challenge of physical autoresearch

As illustrated in Figs. 5 and 3, all coding agents successfully solve the Push-T task in simulation using heuristic learning. However, the real-world environment proves significantly more challenging, causing two of the

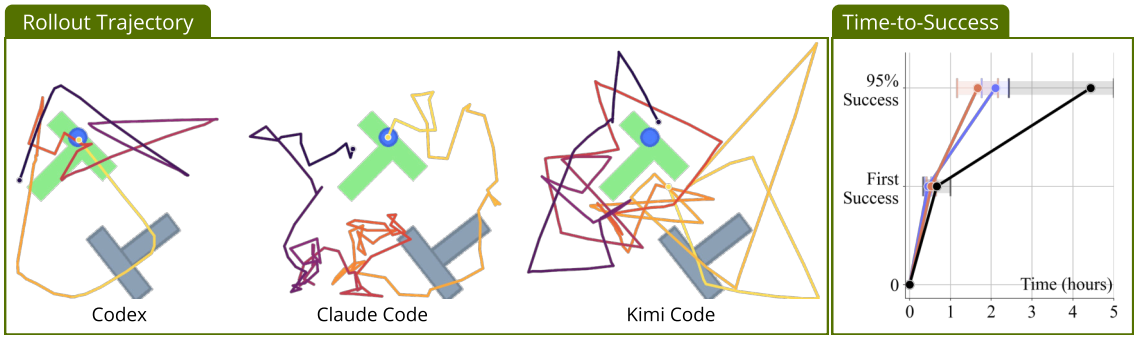


Figure 5: **Autonomous heuristic learning in simulation**. On the Gym-PushT (10) benchmark, Claude Code (orange) and Codex (blue) achieve 95% success rate within approximately 2 hours, while Kimi Code (black) takes twice the time.

three agents to fail. While simulators offer consistent, predictable physics for low-variance hypothesis testing, real-world conditions are non-deterministic and time-varying: factors such as robot dynamics, contact friction, and object movements are inherently less predictable and vary across trials and hardware.

To improve real-world robustness, in subsequent tasks, we encourage agents to explore and even combine diverse learning methods that span heuristics and gradient-based learning, to tackle the corner cases in real-world deployment. We also propose scaling up a physical agent team to test hypotheses in diverse and nondeterministic real-world physics. We will elaborate on these designs in subsequent sections.

3.2. Autoresearch for Gradient-Based Policy Improvement

Apart from supporting heuristic learning, **ENPIRE** is also capable of training end-to-end policies in a precision-critical task, pin insertion, as shown in Fig. 3. In this task, coding agents are required to achieve 50 consecutive successes in real-world evaluations. The agents tested multiple methods to improve the policy, including behavior cloning (BC), iterative BC with online rollout data aggregation, and online, offline, and offline-to-online RL with a BC regularization term. The agents also tuned parameters such as batch size, the actor-critic policy update rate, and the BC-term hyperparameters.

3.3. Scaling Policy Learning on a Robot Fleet

We further study whether scaling the number of robots and agents reduces the wall-clock time required to achieve the same target task performance. We assign one coding agent to one robot, and set up identical environment interfaces, reward functions, and reset mechanisms on this robot fleet. We consider fleet sizes of 1, 4, and 8 robots in Push-T and pin insertion tasks.

In Push-T, scaling from one to eight agents reduces the time to reach a 1.0 normalized score from roughly five to two hours. In pin insertion, scaling from one to eight agents reduces the time to reach a near-perfect success rate from more than 1.5 hours to approximately 40 minutes. This shows that **ENPIRE** is capable of transferring additional robot resources to faster policy improvement through distributed hypothesis selection.

In a multiagent setting, **ENPIRE** can also leverage code-based policies to automatically apply domain randomization during reset. For example, the variations in spatial configurations during GPU insertion span a significantly broader range than prior work (48), which enforces stronger policy robustness.

3.4. Transferring Autoresearch Experience through Agentic Continue Learning

We observe that the insights accumulated by multi-agent physical autoresearch are also transferable to similar, novel dexterous tasks. Following the autonomous exploration phase in pin insertion, agents are prompted

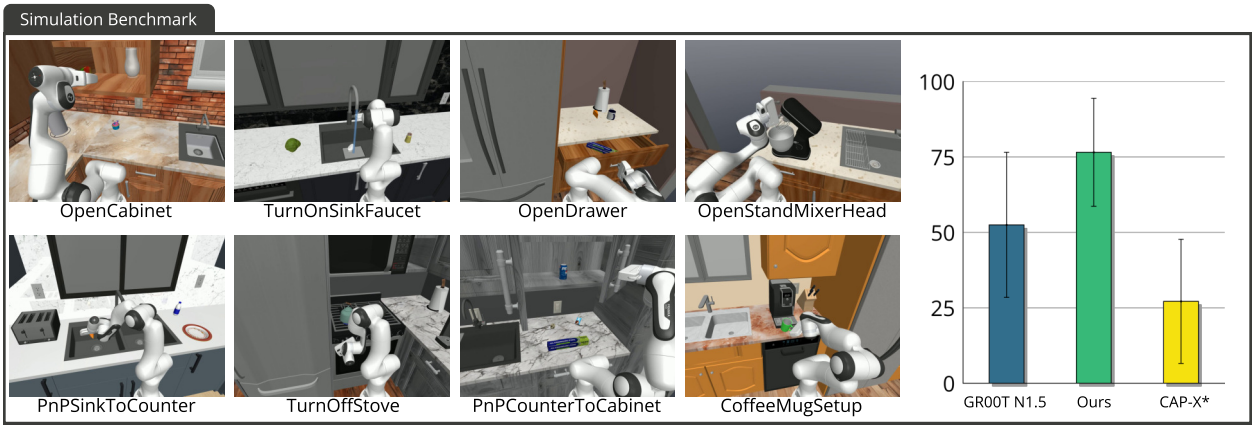


Figure 6: **Simulation results**. On RoboCasa365 (34) benchmark, **ENPIRE** outperforms end-to-end VLA (GR00T (5)) and zero-shot agentic tool use without autoresearch (CaP-X (15)).

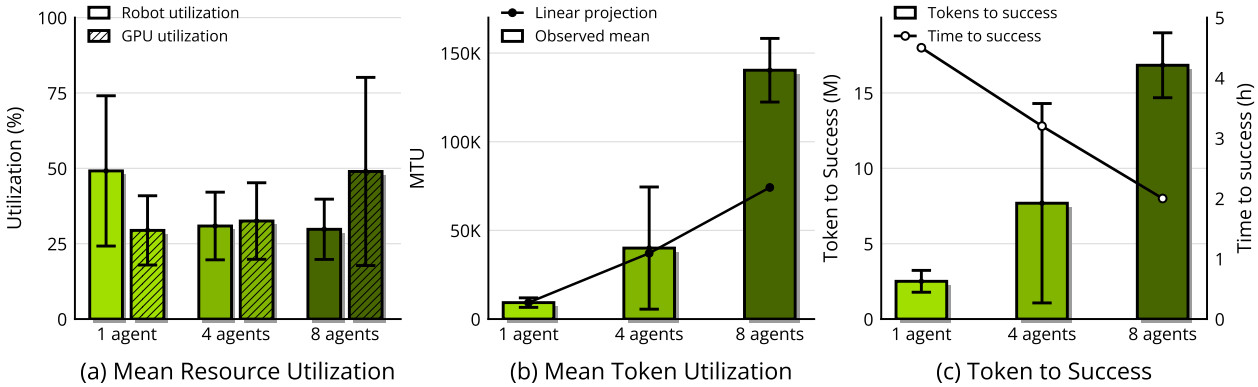


Figure 7: **Quantifying agent resource utilization**. Scaling from 1 to 8 agents raises GPU utilization and token consumption but lowers per-robot utilization.

to document and reflect on the evolution of their training recipes. When we instantiate a new round of autoresearch for the GPU insertion task, appending this knowledge to the new task's instructions allows coding agents to achieve a high success rate. A detailed analysis is provided in the Appendix.

3.5. ENPIRE Discovers Synergy between Code-Based Policies and VLAs

Beyond end-to-end or heuristic training, **ENPIRE** can automatically integrate vision-language-action models (VLAs) (8) with procedural tool calls for long-horizon manipulation. In the RoboCasa365 simulator (34), the agent boosted the success rate of the GR00T VLA (5) by using motion planning and detection tools to hover above an object before grasping. We successfully transferred this strategy to the real world, where the agent learned to hover over scissors, grasp them, and cut a zip tie, as shown in Fig. 2.

3.6. Quantifying Agent Resource Utilization

Fig. 7 summarizes the resource-scaling behavior of ENPIRE on the pin insertion task across single-, four-, and eight-agent fleets. We report Mean Robot Utilization (MRU) and GPU utilization using the definitions introduced above. In addition, we measure Mean Token Utilization (MTU), defined as the number of tokens consumed per minute. We further measure Tokens to Success and Time to Success, which quantify the token budget and wall-clock time required to complete the autoresearch objective.

4. Limitations

Robot and compute resources are underutilized

Coding agents do not fully utilize robot resource when they are reading logs, writing code, debugging, or waiting for the language-model backbone. Moreover, as we scale the number of robots, MRU decreases, while GPU active utilization increases (Fig. 7a). Compared to a single-robot setup, the composition of agent activity causes agent teams to spend more of their time summarizing peer branches and less time actually operating the robot. Coding agents may also fail to launch parallel training sessions to exhaust GPU resources.

Token cost grows super-linearly with fleet size.

As the fleet size increases, token usage grows faster than the ideal linear trend. MTU remains close to the linear projection up to four agents, but rises sharply at eight agents (Fig. 7b). The total token budget required to obtain a successful policy shows the same trend, increasing much more rapidly than the corresponding reduction in wall-clock time (Fig. 7c). Thus, increasing the fleet size trades token efficiency for faster policy improvement: larger fleets reach success sooner, but require a disproportionately higher token budget.

5. Related Work

5.1. Coding Agents & Code as Policies

This line of work rests on a central abstraction—*executable code as the agent's action*: the model generates a program, the runtime returns structured feedback, and the loop iterates. Code-as-Policies (25) and ProgPrompt (41) introduced this formulation for robotics, composing perception and skill APIs into a task plan, and visual reasoning carried the same compositional idea to vision modules (42). Subsequent research extended single-shot generation into multi-turn loops driven by execution feedback—reason-and-act traces (51), self-critique and iterative repair (32; 40), inference-time search (52), and learned tool invocation (37); Wang et al. (45) argues code is a stronger action representation than language because each step is runtime-verifiable. A parallel thread formalizes the *agent–computer interface*, a sandboxed shell in which agents read, run, and debug code, now underlying frontier coding products (3; 33; 35).

Within robotics, early code-as-policy systems exposed high-level, human-engineered APIs—closed-form skills, language-conditioned grasping primitives, and affordance scorers (1)—leaving the agent to decompose tasks rather than synthesize low-level control. Wang et al. (44) grows a reusable Minecraft skill library via self-verification and an automatic curriculum where trials are near-free, and Fu et al. (15) finds that multi-turn feedback, skill synthesis, and ensembled sampling improve manipulation reliability over low-level primitives. A related line uses LLMs to synthesize auxiliary training signals: reward functions (31; 49; 53), sim-to-real transfer protocols (30), simulation environments (46), and data-collection pipelines (2).

5.2. Agentic Self-Improvement

A self-improvement loop needs each trial to be cheap enough to repeat at scale; systems differ mainly in what a trial returns to the agent. Ellis et al. (13) set the retention pattern that the rest build on: each solved synthesis task adds named subroutines to a growing library, which is viable because execution is free. Wang et al. (44) carried this into an embodied setting, swapping the success signal for LLM self-verification and an automatic curriculum—again viable because Minecraft rollouts cost nothing. Yu et al. (53), Ma et al. (31), and Xie et al. (49) return a reward rather than a skill: an LLM proposes a dense reward, a policy is trained against it, and the reward is revised from training statistics—a loop Eureka closes via thousands of Isaac Gym rollouts per minute. Ma et al. (30) reaches toward hardware via synthesized domain-randomization parameters, yet still iterates in simulation and deploys only after revision stops; at the experimental-setting level, Wang et al. (46) generates new tasks and assets in simulation, and Ahn et al. (2) coordinates a mobile-robot fleet for offline data collection rather than within-loop iteration. In every case, the loop closes on a cheap substrate while

real-robot execution serves only as a sim-to-real or data target, never as the medium of iteration. We retain these skill-accumulation and reward-generation mechanisms but run the loop directly on hardware, where the binding resource is the agent's robot-access budget, not its compute.

5.3. Autonomous Research Agents and Scientific Discovery

The final body of work automates the research loop itself, best read along *the medium in which experiments are launched*. Prior to LLMs, autonomous systems closed the hypothesis–experiment loop on real laboratory hardware (9; 23). LLM-era systems instead automate the *digital* loop end to end (26; 38; 39; 50), with a recent strand reaching back into the physical sciences through lab automation (6; 29) or human-executed wet-lab validation (16). Evaluation tracks this digital emphasis: MLE-bench (11) scores ML-engineering agents and resource scaling, while analyses of SWE-bench (21) caution that gains can reflect contamination over capability. Two gaps follow: no system autonomously runs and optimizes a physical *robotics* loop under an explicit hardware budget—classical robot scientists fixed the apparatus and never wrote their own tools, while LLM research agents never touch real robots—and no benchmark measures utilization of a scarce physical resource rather than capability or cost-per-paper. Our system addresses both gaps by running and optimizing the experimental loop on real robots and introducing resource-utilization metrics for this budget-bound regime.

6. Acknowledgement

We are grateful to many colleagues whose help made this project possible. We thank Jason Liu, Tony Tao, Tairan He, Alex Lin, Jim Yang, Paul Zhou, and Abhi Maddukuri for insightful discussions and feedback; Yide Shentu, Bike Zhang, Angchen Xie, Dvij Kalaria, and Yuqi Xie for their support with the experiments; Lion Park, Matin Furutan, Jeremy Chimienti, Dennis Da, and Tri Cao for fleet operation; and Tri Cao for the demo shots. We also thank the NVIDIA GEAR Team and the CMU LeCAR Lab for their continuous support.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

[2] Michael Ahn, Debidatta Dwibedi, Chelsea Finn, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Karol Hausman, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, et al. Autort: Embodied foundation models for large scale orchestration of robotic agents. *arXiv preprint arXiv:2401.12963*, 2024.

[3] Anthropic. Introducing claude opus 4.7. https://www.anthropic.com/news/claude-opus-4-7, 2026.

[4] Philip J Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, pages 1577–1594. PMLR, 2023.

[5] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[6] Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023.

[7] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

[8] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.

[9] Benjamin Burger, Phillip M Maffettone, Vladimir V Gusev, Catherine M Aitchison, Yang Bai, Xiaoyan Wang, Xiaobo Li, Ben M Alston, Buyi Li, Rob Clowes, et al. A mobile robotic chemist. *Nature*, 583(7815): 237–241, 2020.

[10] Rémi Cadène, Quentin Gallouédec, Alexander Soare, and Simon Alibert. gym-pusht: A gymnasium environment for PushT. https://github.com/huggingface/gym-pusht, 2024. Version 0.1.6, adapted from Diffusion Policy.

[11] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, volume 2025, pages 50466–50494, 2025.

[12] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

[13] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*, pages 835–850, 2021.

[14] Martin A Fischler and Robert C Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.

[15] Max Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Fei-Fei Li, Guanya Shi, et al. Cap-x: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026.

[16] Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J Szostkiewicz, Dmytro Shved, Gavin J Gyimesi, Jon M Laurent, Samantha M Wright, Muhammed T Razzak, et al. A multi-agent system for automating scientific discovery. *Nature*, pages 1–3, 2026.

[17] Stefan Gottschalk, Ming C Lin, and Dinesh Manocha. Obbtree: A hierarchical structure for rapid interference detection. In *Proceedings of the 23rd annual conference on Computer graphics and interactive techniques*, pages 171–180, 1996.

[18] Tuomas Haarnoja, Sehoon Ha, Aurick Zhou, Jie Tan, George Tucker, and Sergey Levine. Learning to walk via deep reinforcement learning. *arXiv preprint arXiv:1812.11103*, 2018.

[19] Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, et al. pi*0.6: a vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.

[20] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. pi05: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[21] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157, 2024.

[22] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. https://github.com/karpathy/autoresearch, 2025. GitHub repository.

[23] Ross D King, Kenneth E Whelan, Ffion M Jones, Philip GK Reiser, Christopher H Bryant, Stephen H Muggleton, Douglas B Kell, and Stephen G Oliver. Functional genomic hypothesis generation and experimentation by a robot scientist. *Nature*, 427(6971):247–252, 2004.

[24] Kun Lei, Huanyu Li, Dongjie Yu, Zhenyu Wei, Lingxiao Guo, Zhennan Jiang, Ziyu Wang, Shiyu Liang, and Huazhe Xu. Rl-100: Performant robotic manipulation with real-world reinforcement learning. *arXiv preprint arXiv:2510.14830*, 2025.

[25] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*, pages 9493–9500. IEEE, 2023.

[26] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

[27] Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. Serl: A software suite for sample-efficient robotic reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 16961–16969. IEEE, 2024.

[28] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105):eads5033, 2025.

[29] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature machine intelligence*, 6(5):525–535, 2024.

[30] Jason Ma, William Liang, Hung-Ju Wang, Yuke Zhu, Linxi Fan, Osbert Bastani, and Dinesh Jayaraman. Dreureka: Language model guided sim-to-real transfer. RSS, 2024.

[31] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Jim Fan, et al. Eureka: Human-level reward design via coding large language models. In *International conference on learning Representations*, volume 2024, pages 26516–26560, 2024.

[32] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.

[33] Moonshot AI. Kimi code. https://www.kimi.com/code/en, 2026.

[34] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. Robocasa365: A large-scale simulation framework for training and benchmarking generalist robots. *arXiv preprint arXiv:2603.04356*, 2026.

[35] OpenAI. Openai codex. https://developers.openai.com/codex/, 2026.

[36] Dean A Pomerleau. Alvinn: An autonomous land vehicle in a neural network. *Advances in neural information processing systems*, 1, 1988.

[37] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.

[38] Samuel Schmidgall and Michael Moor. Agentrxiv: Towards collaborative autonomous research. *arXiv preprint arXiv:2503.18102*, 2025.

[39] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using llm agents as research assistants. *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 5977–6043, 2025.

[40] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.

[41] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*, 2022.

[42] Dídac Surís, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 11888–11898, 2023.

[43] Simon Thorpe, Denis Fize, and Catherine Marlot. Speed of processing in the human visual system. *Nature*, 381(6582):520–522, 1996. doi: 10.1038/381520a0.

[44] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[45] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*, 2024.

[46] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. Robogen: Towards unleashing infinite data for automated robot learning via generative simulation. *arXiv preprint arXiv:2311.01455*, 2023.

[47] Jiayi Weng. Learning beyond gradients. https://trinkle23897.github.io/learning-beyond-gradients/, May 2026. Blog post.

[48] Wenli Xiao, Haotian Lin, Andy Peng, Haoru Xue, Tairan He, Yuqi Xie, Fengyuan Hu, Jimmy Wu, Zhengyi Luo, Linxi Fan, et al. Self-improving vision-language-action models with data generation via residual rl. *arXiv preprint arXiv:2511.00091*, 2025.

[49] Tianbao Xie, Siheng Zhao, Chen Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. Text2reward: Reward shaping with language models for reinforcement learning. In *International Conference on Learning Representations*, volume 2024, pages 35663–35699, 2024.

[50] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.

[51] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

[52] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.

[53] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, et al. Language to rewards for robotic skill synthesis. *arXiv preprint arXiv:2306.08647*, 2023.

Appendices

All videos are available at research.nvidia.com/labs/gear/enpire

Appendix Outline

A Environment Creation **15**

A.1 Autonomous Reset . . . 15

A.2 Autoresearch-driven Reward Design . . . 16

B Robot System Setup **18**

B.1 Fleet Architecture . . . 18

B.2 Station Hardware . . . 18

B.3 Low-Level Control . . . 19

B.4 Per-Task Configuration . . . 19

B.5 Real-World RL System Integration . . . 20

B.6 Idea Tree for Pin Insertion . . . 20

C Physical Autoresearch: Ablation Studies **21**

C.1 Experimental Setup . . . 21

C.2 Token Utilization . . . 21

C.3 Native Vision Capability . . . 21

C.4 Model and Harness Comparison . . . 21

D Simulation Benchmark Result **21**

D.1 RoboCasa Simulation Interface and API Surface . . . 21

D.2 RoboCasa Evaluation Protocol . . . 23

A. Environment Creation

A.1. Autonomous Reset

We show that, given a task specification, coding agents can synthesize long-horizon reset functions by composing perception and planning APIs, including SAM3 for open-vocabulary object detection (illustrated in Code Snippet 1), BundleSDF for continuous 6-DOF pose tracking, and cuRobo for GPU-accelerated collision-free trajectory optimization. During execution, these reset functions consume real-time RGBD streams and proprioceptive state, which together support robust object localization, grasp pose generation, and motion planning. Coding agents can also command and read gripper torque signals, which serve as a surrogate for tactile feedback; we find that these signals are crucial for slip detection and controlling the force of the grip in precision-critical manipulation tasks.

Code Snippet 1: **Tool API: SAM3 Multi-Object Segmentation.** Detects all instances of a queried object from a chosen camera view, returning per-instance masks, confidence scores, and back-projected 3D centroids.

```python
class SegmentAllObjectsTool(Tool):
    """Detect all instances of an object using SAM3 multi-detection."""
    parameters = [
        ToolParameter("query",                "str"),
        ToolParameter("camera",               "str",   default="top"),
        ToolParameter("score_thresh",         "float", default=0.1),
        ToolParameter("image_bbox",           "list",  default=None),
        ToolParameter("image_exclude_bboxes", "list",  default=None),
        ToolParameter("filters",              "list",  default=None),
        ToolParameter("rank_by",              "dict",  default=None),
    ]
    # query:
    #     Natural-language description of the object to segment.
    # camera:
    #     Camera view to capture from. One of {top, left, right}.
    # score_thresh:
    #     Minimum SAM3 confidence score. Detections below threshold are dropped.
    # image_bbox:
    #     [x_min, y_min, x_max, y_max] pixel crop. Zeros all pixels outside the
    #     box before sending to SAM3 to suppress background noise. Returned masks
    #     remain in full-image coordinates so centroid_world_xyz back-projects correctly.
    # image_exclude_bboxes:
    #     List of [x_min, y_min, x_max, y_max] boxes to zero out before sending to
    #     SAM3. Inverse of image_bbox — used to blank known distractors (e.g. gripper
    #     jaws). Combinable with image_bbox: image_bbox is applied first, then
    #     exclusions zero out within the kept region.
    # filters:
    #     List of {metric, op, value} dicts AND-ed together to prune detections.
    #     metric in {confidence, area, AR, C, P}; op in {>=, <=}.
    #     AR: aspect ratio; C: compactness 4*pi*area/P^2; P: perimeter in pixels.
    #     Detections with None or non-finite metric values are dropped.
    # rank_by:
    #     {metric, direction} dict controlling sort order of returned detections.
    #     metric in {confidence, area, AR, C, P}; direction in {max, min}.
    #     Default: {confidence, max}.

    def execute(self, **kwargs) -> ToolResult:
        rgb         = self._fetch_camera_image(kwargs["camera"])
        rgb_for_sam = self._preprocess(rgb, kwargs["image_bbox"],
                                       kwargs["image_exclude_bboxes"])
        detections  = requests.post(f"{self._sam3_url}/segment_all",
                          json={"text":            kwargs["query"],
                                "image_b64":       to_b64(rgb_for_sam),
                                "score_threshold":kwargs["score_thresh"]}).json()
        depth, intr, T_cam_world = _fetch_camera_params(self._cam, kwargs["camera"])
        results = [
            SegmentationResult(
                mask               = np.load(io.BytesIO(base64.b64decode(d["mask_b64"]))),
                score              = d["score"],
                bbox_xywh          = d["bbox_xywh"],
                mask_area          = d["mask_area"],
                perimeter_px       = _shape_stats(mask)["perimeter_px"],
                compactness        = _shape_stats(mask)["compactness"],
                centroid_uv        = _shape_stats(mask)["centroid_uv"],
                centroid_world_xyz = _project_centroid(mask, depth, intr, T_cam_world),
                depth_m            = depth[v, u],
            ) for d in detections
        ]
        return ToolResult(success=True,
                          data=_rank_results(_apply_filters(results,
                                             kwargs["filters"]), kwargs["rank_by"]))
```

From these API tools, coding agents can synthesize perception skills such as semantic segmentation and 3D bounding box generation, as well as manipulation skills including pick-and-place, object reorientation, localization, and bimanual handover. Composing these skills with failure detection and retry mechanisms yields long-horizon reset functions that are robust to real-world perturbations. Code Snippet 2 illustrates a concrete example for the GPU insertion task, where the agent sequences SAM3-based object localization, RANSAC (14) and OBB (17)-based 3D bounding box estimation, torque-verified grasping, and cuRobo collision-free handover into a complete reset pipeline. Visualizations of the intermediate segmentation and bounding box outputs are provided in Fig. 8.

Code Snippet 2: **Reset Policy: GPU Insertion Recovery.** Localizes the motherboard and target slot with SAM3, grasps and extracts the GPU with a torque-verified grip, then carries it to a parking pose via a cuRobo collision-free handover.

```python
def reset_gpu_insertion():
    go_home()

    # --- SAM3: localize motherboard and target PCIe slot ---
    scene = sam3_localize_motherboard(camera="top")
    hover_pos, scene = acquire_slot_hover(scene)

    # --- grasp GPU and extract from slot ---
```

```
    descend_guided(scene)
    close_gripper(torque_limit=LEFT_CLOSE_TORQUE_LIMIT)
    verify_grasp_or_raise()                                  # gripper torque check
    lift_and_pull(scene)

    # --- handover: carry GPU to parking pose on table ---
    freespace_move(left_target_pos=TABLE_APPROACH_POS,
                   left_target_rpy=TABLE_PLACE_RPY,
                   backend="curobo")
    release_glide(TABLE_PLACE_POS, TABLE_PLACE_RPY)     # synchronized open
    cartesian_retract_after_release()
    go_home()
```



Figure 8: Visual tools for GPU insertion. Agent-written auxiliary detection tools for GPU localization via SAM3 and depth estimation.

A.2. Autoresearch-driven Reward Design

Autoresearch can be applied to automate the design of reward functions. This procedure requires human effort in only two steps: first, collect a few minutes of representative success and failure demonstrations; second, specify the reward design requirements, including target precision, recall, and inference latency.

As an example, we assign the agent to build a visual reward function that detects whether a zip tie is fastened. For evaluation, we record a few minutes of labeled success and failure snapshots of zip tie fastening as a sandboxed held-out evaluation set—the agent may study failures, but cannot train on the test set or alter metric computation. The agent autonomously probes the SAM3 API via prompt search, tunes cropping and depth thresholds to suppress background noise, and meets the 150 ms latency budget by converting to ONNX, compiling the computation graph for simultaneous multi-object forward passes. The decision-making logic converges on the two-view geometric test, illustrated in Fig. 4. A simplified implementation is provided in Code Snippet 3.

Autoresearch is not limited to visual rewards; the coding agent can compose hybrid reward functions that fuse multiple sensing modalities, including proprioception and end-effector force estimates. As an example, Fig. 9 illustrates an autonomously designed reward for pin insertion that combines visual alignment, depth of insertion, and contact force signals.

Code Snippet 3: **Dual-View Zip Tie Verification.** Combines the top and right camera views in a geometric test of whether the zip-tie strap passes through the head, guarding against single-view false positives.

```
def get_reward_from_top_right_cam(rgb_t, rgb_r):
    dets = segment_zip_tie_views(rgb_t, rgb_r, threshold=sam3_score_threshold)
    strap = union([d["mask"] for d in top2_by_score(dets["top"]["strap"])])
    head = top1_by_score(dets["top"]["head"])["mask"]
    strap = binary_dilation(strap, iterations=top_dilate)
    head = binary_dilation(head, iterations=top_dilate)
    overlap = (strap & head).sum() / head.sum()
    protrude = strap[:, :head_left_x(head)].sum() / head.sum()
    top_ok = overlap >= top_inter_frac and protrude >= top_left_mult

    right = binary_dilation(dets["right"]["strap"]["mask"], iterations=right_dilate)
    head_box = bbox_shrunk(dets["right"]["head"], frac=right_shrunk_frac)
    labeled, n_comp = label(right)
    if n_comp == 2:
        right = right | connect_components(labeled)
    right_ok = n_comp in (1, 2) and (right & head_box).any()
    return top_ok and right_ok
```

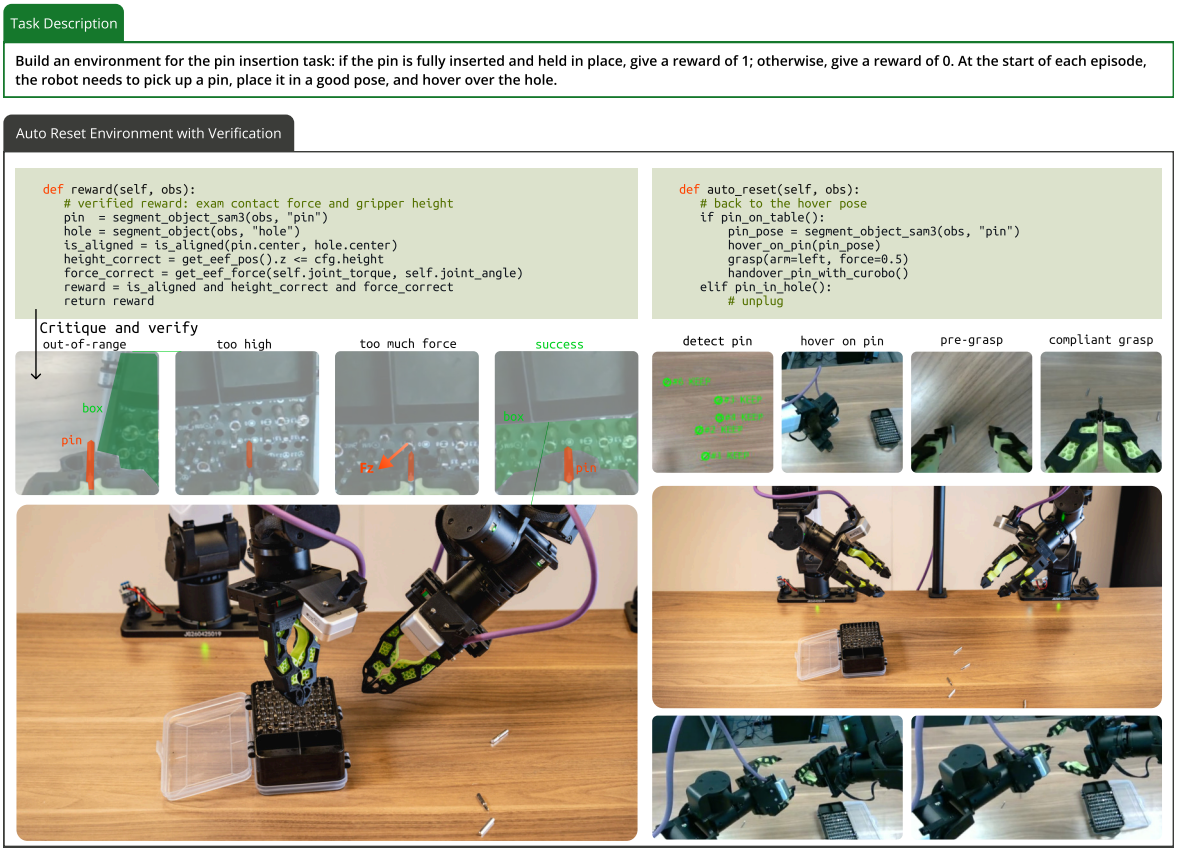


Figure 9: Reward function for pin insertion. The agent proposes a hybrid verification strategy fusing visual alignment of the pin tip to the hole, insertion depth from proprioception signals, and end-effector torque detection.

B. Robot System Setup

B.1. Fleet Architecture

Our physical infrastructure is a fleet of **eight bimanual robot stations**, operated in a decentralized manner. Each station owns its robot arms and cameras, compute, and its own coding agent. Hardware-control requests from each station's agent are routed through a local FastAPI server. Coordination across stations is mediated entirely through **Git**: rather than streaming state to a central server, stations share code, configurations, tools, and results by pushing to and pulling from a common repository, so that improvements discovered on one station propagate to the rest through ordinary version control. Because all coordination flows through Git, agents can borrow ideas from one another: they freely merge changes from other branches or cherry-pick individual commits, selectively adopting promising approaches and results discovered at other stations. This keeps the fleet loosely coupled and fault-tolerant, since stations run, fail, and recover independently, while the shared Git history serves as the single source of truth for what each station has tried and learned.

Control endpoints.
The FastAPI server exposes a small set of endpoints through which the agent drives data collection and execution: `/start` begins a rollout on the real hardware, `/restart` allocates a fresh rollout buffer directory, and `/home` returns the arms to the home configuration. The `/restart` endpoint keeps successive rollouts separate and individually addressable, so that the data from different hypotheses or experiments is never mixed together: each rollout is written to its own directory, and the agent can attribute outcomes to the specific change it was testing. These endpoints are shared across all of our tasks. The Push-T task additionally exposes `/avoid` and `/resume` endpoints to handle occlusion: `/avoid` backs the arm out of the top-camera frame so it no longer occludes the scene, and `/resume` returns the arm to its original position to continue manipulation.

```
/goal Achieve a 100% success rate over each window of 50 consecutive
episodes. Read @auto_research.md and fan out a subagent team to study
different algorithms (RL, BC, hybrid). You may reuse checkpoints or data
buffers across hypotheses. Create your own branch from autorl following the
instructions. Do not directly work on or push to autorl. You must actively
monitor other branches and commits from origin that branched off autorl to
leverage knowledge from those branches. You must actively push your branch to
contribute to the repo and pull from origin. Collaborate with other agents
via Git.
```

Figure 10: The autoresearch prompt provided to each station's coding agent.

Agent sandbox and context.
Each station's coding agent operates within a sandbox scoped to a single autoresearch repository. Within this sandbox the agent runs with elevated autonomy: action-level permission prompts are bypassed, so it can execute commands without per-step human approval, and it is granted unrestricted internet access. The agent is provided with the complete set of robot data collected during the current autoresearch session, and is free to make full use of all available information in pursuit of its objective. For ordinary fresh sessions, raw repository state, rollout data, checkpoints, and transient logs from prior sessions are pruned before initialization, so that the agent starts from a clean, isolated workspace. For transfer experiments, such as initializing GPU insertion after pin insertion, the agent is instead given an explicit markdown summary distilled from the previous pin-insertion autoresearch session; raw trajectories, hidden logs, and checkpoints are still removed, so transfer occurs through the written summary rather than through unbounded access to prior session state. The prompt given to each station's agent is shown in Fig. 10.

B.2. Station Hardware

All 8 stations are hardware-identical. Each station comprises two YAM (Yet Another Manipulator) arms from I2RT in a fixed bimanual configuration, a set of cameras, and a single workstation that runs the FastAPI server, policy inference, and the station's agent.

Manipulators and actuation.
Each arm is a 6-DoF manipulator equipped with a 1-DoF parallel-jaw gripper, giving seven actuated joints per arm and fourteen across the bimanual pair. All joints are driven by brushless actuators over a CAN bus. The six arm joints are run under PD control with gravity compensation, while the gripper is run in a force-limited mode that bounds grip force directly (Sec. B.3); we exploit this gripper-level force limiting both for robust grasping and for safe, unattended operation across the fleet.

Compute.
Each station runs on a single workstation whose specifications are listed in Tab. 1. All policy inference and on-station computation run on this single GPU, with no shared cluster or off-station compute.

Table 1: Per-station compute specifications.

Component	Specification
GPU	1× NVIDIA RTX 5090, 32 GB
CPU	Intel Core Ultra 9 285K, 24 cores
RAM	128 GB
OS	Ubuntu 22.04 LTS
GPU stack	NVIDIA driver 595.58.03, CUDA 13.2

Perception.
Unless noted otherwise, perception uses Intel RealSense **D405** cameras: one top-down camera viewing the workspace and two wrist-mounted cameras, one per arm, that provide close-range views of the grasped object and the contact region. One task also uses a side-mounted RealSense **D435i** for a wider third-person view (Sec. B.4). The policy runs at **30 Hz**, and its action targets are tracked by the low-level joint controllers described below, which run at **100 Hz** over the CAN bus. We additionally use **Viser** for real-time, browser-based 3D visualization of robot state, camera frames, and target poses, which supports monitoring of autonomous runs, calibration debugging, and episode inspection.

B.3. Low-Level Control

Policy actions are inferred at **30 Hz**, and each action target is tracked by the low-level joint controllers running at **100 Hz** over the CAN bus. The two joint groups are controlled differently. The six arm joints use **PD control with gravity compensation**, which tracks the commanded joint targets while the feedforward gravity term offloads the static load so the PD gains only need to handle the residual error.

The 1-DoF gripper runs as a **torque-limiting compliant grasp**: instead of closing to a rigid target width, the gripper applies a bounded grip force set by a commanded torque limit. This force limiting is the key to robust and safe grasping. Because the fingers settle around the object at a bounded force rather than driving to a fixed position, the grasp accommodates variation in object pose and size and is robust to perception and placement error. The same bound caps the force the system can exert during grasping, contact, and insertion, preventing damage to the gripper, the manipulated parts, and the fixtures when an attempt is misaligned or fails. Because the fleet operates autonomously and unattended across all 8 stations, this bounded-force behavior is essential: a bad contact results in a safe stall rather than a hardware-damaging push, with no human in the loop to intervene.

B.4. Per-Task Configuration



(a) Four-camera setup (GPU insertion).



(b) Three-camera setup (pin insertion, Push-T, zip tie).

Figure 11: Per-station camera configurations. All tasks use one top-down camera and two wrist-mounted cameras, all RealSense D405. The GPU insertion task in Fig. 11a additionally uses a side-mounted RealSense D435i on the left pole for a wider third-person view of the slot; the remaining tasks use the three-camera setup in Fig. 11b.

The four tasks share the station hardware and the 30 Hz policy loop above, and differ only in minor details of their camera and control configuration. As shown in Fig. 11, all four use one top-down camera and two wrist cameras, all RealSense D405; the GPU insertion task additionally uses a side-mounted RealSense D435i for a wider third-person view of the slot.

B.5. Real-World RL System Integration

We integrated the PLD-RL pipeline (48) to provide a controlled sandbox for the coding agent to conduct algorithm auto-research with online data collection. The infrastructure is developed following the asynchronous design of SERL (27; 28). We implement it as a three-tier distributed system that decouples robot interaction, policy learning, and inference across separate processes. The deployment layer runs on a robot-adjacent control machine and is responsible for hardware orchestration, episode recording, and human-in-the-loop teleoperation. The second tier, learner, runs on a GPU host and trains an RL agent (actor & critic) with pixel observations encoded through a pre-trained visual backbone. The third tier, actor, exposes a Portal/ZMQ msgpack-compatible endpoint, so that the controller can request actions through a uniform protocol shared with scripted and teleoperated control modes.

Data flow between the deployment layer and the learner is mediated by a deliberately loose disk-based contract. The deployment layer serializes each episode as a per-step observation tensor and synchronized camera streams (.mp4) into a rollout-buffer directory, accompanied by a per-step action label identifying the action source (rl, human, etc.). Meanwhile, a daemon thread (*DiskBufferIngestor*) polls this directory periodically, parses newly finalized episodes, and routes transitions by action-source label following the RLPD-style data-mixing protocol (4): RL-generated transitions populate the online replay buffer, while human and manual transitions populate a separate demonstration buffer that is mixed into each training batch. This design is flexible for mixing data from different sources.

B.6. Idea Tree for Pin Insertion

Fig. 12 visualizes an agent-team autoresearch run on the pin insertion task as an idea tree (top) paired with its hill-climbing curve (bottom). Each node Ik is an idea explored by the team; ideas that are related are connected by a horizontal line, and each new lane corresponds to a new branch of ideas. Solid green nodes mark ideas that improved the team-average best success rate, while hollow nodes were evaluated but yielded no gain. The thick black line traces the lineage of the highest-scoring idea. Circled nodes are highlighted ideas that are also annotated on the hill-climbing curve below, to which they are connected by dashed lines. The bottom panel reports the team-average best success rate as a function of research wall-clock time, sharing the time axis of the tree above. As the agents collaborate through Git, a few high-impact ideas account for most of the progress—most notably BC regularization (I37, $+10.8$ pp)—while later ideas such as batch-size tuning (I66, $+0.9$ pp) and controller compensation (I76, $+1.3$ pp) contribute smaller, incremental refinements as the success rate approaches 100%.

C. Physical Autoresearch: Ablation Studies

C.1. Experimental Setup

We constructed a simplified real-world Push-T environment, `pusht-simple`, for controlled ablations of the autoresearch agent. The task preserves the core visual servoing and contact-planning structure of Push-T while reducing episode length and setup variability, making it suitable for repeated comparisons across model, harness, and visual-grounding conditions. Compared with the Push-T task in the main paper, this variant uses a relaxed success boundary so that ablations can focus on controller discovery rather than rare terminal precision. An episode is counted as successful when the pushed T block reaches the target region within the prescribed translational tolerance and its orientation is within 10° of the goal orientation. Operationally, the agent must write a controller that rotates the T block by approximately 180° and places it inside the enlarged success region.

C.2. Token Utilization

In the main paper, we summarize token scaling by aggregating input and output tokens. Because input, output, and cached tokens differ in cost, latency, and the type of agent activity they reflect, we additionally report token usage by category. This breakdown clarifies how much of the autoresearch budget is spent on context ingestion, code and plan generation, and reuse of cached interaction history. For a more detailed qualitative analysis, we visualize total token usage, input token usage, cached input token usage, fresh input token usage, output token usage, and reasoning token usage.

C.3. Native Vision Capability

Since Push-T requires the agent to localize the object, infer contact geometry, and diagnose failures from evaluation rollouts, we ablate the effect of native vision capability in the coding agent. The Codex harness supports native visual understanding, meaning that the coding agent can directly inspect images. However, many open-source agent harnesses expose only text input. This experiment measures how visual access affects auto-search hill-climbing on the simplified Push-T task. In addition to the original Codex configuration, we evaluated two baselines:

1. **Codex without native vision;** We mask image tokens from the coding agent but provide a separate image-understanding module as a callable function. The module reads images, produces descriptions, and answers visual questions; the system prompt is modified to allow the coding agent to call this function when visual information is needed.
2. **Codex without visual capability.** We remove both native image streaming and visual function calling. In this setting, Codex can only analyze text-based information or write code to extract information from images.

Codex with native vision reaches success first. Surprisingly, the no-vision baseline succeeds before the function-call vision baseline. This suggests that even without direct visual access, the coding agent can infer useful task state from other logging signals, while repeated image function calls introduce additional overhead during task solving.

C.4. Model and Harness Comparison

We study how the choice of model and agent harness affects end-to-end autoresearch performance. We compare Codex with GPT-5.5, Claude with Opus 4.7, and Codex with Opus 4.7. Codex solves the task fastest, while the Codex harness using the Opus API is the least efficient among the tested configurations.

D. Simulation Benchmark Result

D.1. RoboCasa Simulation Interface and API Surface

The RoboCasa365 benchmark provides the simulation-side evaluation setting for generated vision-language control scripts. We build middle-level vision and planning stacks on top of the RoboCasa simulation, and encapsulate modular and end-to-end tools as APIs. This interface is injected into generated vision scripts as a single Python namespace. For canonical evaluation scripts, APIs that leak privileged state or enable repeated retries are removed from the runtime namespace. In particular, the environment disables APIs such as `get_task_info`, `reset_env`, `reset_to_initial`, task-specific CLI helpers, and oracle-target queries. The oracle target API is only exposed outside the canonical script runtime. Tab. 2 summarizes the surface and marks, for each call, whether it is available in the canonical script runtime (✓) or gated to CLI/oracle modes (C/O).

Table 2: RoboCasa environment API surface used in our evaluation. *Avail.* indicates availability in the canonical script runtime: ✓ available; C exposed only in non-canonical CLI mode; O exposed only when the Oracle interface is explicitly enabled.

API	Description	Avail.
State and gripper		
`get_robot_state`	Joint positions, EEF pose, gripper measurement, and base/arm state	✓
`get_gripper_info`	Gripper command, measured state, contact bodies, and actuator force	✓
`set_gripper`	Set normalized gripper command (1 open, 0 closed)	✓
`open_gripper`	Open the gripper	✓
`close_gripper`	Close the gripper	✓
Motion		

API	Description	Avail.
`move_with_curobo`	cuRobo plan, OSC waypoint tracking, scene collision	✓
`move_with_pyroki`	Pyroki IK with SE(3) interpolation, no scene collision	✓
`move_with_curobo_joint_exec`	cuRobo plan executed in joint-position mode	✓
`move_with_pyroki_joint_exec`	Pyroki plan executed in joint-position mode	✓
`freespace_move`	Backward-compatible wrapper over configured backend	✓
`move_eef`	Move to a direct end-effector target	✓
`select_best_grasp`	Planner-check grasp candidates and select a feasible one	✓
`go_home`	Return to the home configuration	✓
`nudge`	Small local end-effector displacement	✓
`nudge_brutal`	Small local move with collision avoidance off	✓
`rotate_gripper_in_place`	In-place wrist rotation	✓
Camera and visualization		
`get_camera_image_480_640`	RGB image at 480×640	✓
`get_camera_depth_480_640`	Aligned depth map at 480×640	✓
`get_camera_intrinsics_480_640`	Intrinsics matched to 480×640	✓
`get_camera_image`	Legacy RGB image alias at the environment-native resolution	✓
`get_camera_depth`	Legacy depth alias at the environment-native resolution	✓
`get_camera_intrinsics`	Legacy intrinsics alias at the environment-native resolution	✓
`get_camera_extrinsics`	Camera extrinsics	✓
`set_debug_markers`	Place visualization markers	✓
Perception		
`detect_object`	Text-conditioned object detection	✓
`segment_object`	Single-instance segmentation	✓
`segment_object_all`	Multi-instance segmentation	✓
`detect_objects_oneshot`	One-shot object detection	✓
`sample_grasp_pose_anygrasp`	AnyGrasp grasp-pose sampling	✓
`vlm_query`	Vision-language model query	✓
Planner and collision		
`update_planner_world`	Set the cuRobo collision geometry	✓
`refresh_planner_world`	Refresh collision geometry from the scene	✓
`disable_collision_avoid`	Disable collision avoidance	✓
Task, policy, and navigation		
`get_task_description`	Natural-language task instruction	✓
`use_policy_output`	Query an external policy's output	✓
`get_base_state`	Mobile-base state	✓
`execute_base_trajectory`	Execute a planned base trajectory	✓
`create_goat_agent`	Instantiate a GOAT navigation agent	✓
`set_goat_goal`	Set the GOAT navigation goal	✓
`goat_navigate`	Run GOAT navigation	✓
`get_goat_state`	Return GOAT state and configuration	✓
`reset_goat`	Reset GOAT state	✓
Utilities		
`display_rpy_to_quat`	Convert display RPY angles to an `xyzw` quaternion	✓
`np`, `numpy`	NumPy convenience exports for generated scripts	✓
Gated / removed in the canonical script runtime		
`get_task_info`	Structured task information	C
`reset_env`	Reset the environment	C
`reset_to_initial`	Reset to the initial state	C
`get_oracle_targets`	Ground-truth target pose / oracle query	O

Camera and visualization.
The camera APIs provide RGB-D observations, camera intrinsics, and extrinsics. We use resolution-specific calls to keep RGB, depth, and intrinsics mutually aligned, while retaining unsuffixed aliases for backward compatibility. Debug markers are used only for visualizing intermediate targets during development.

Motion.
The motion APIs provide both cuRobo- and Pyroki-based execution. cuRobo uses scene collision geometry and is the default choice for free-space manipulation, whereas Pyroki provides lightweight SE(3) interpolation with inverse kinematics but no scene-level collision model. The standard mode tracks end-effector waypoints with OSC, while joint-execution variants are used when faithfully following the planned joint path is important, such as near joint limits or tight obstacles.

Perception.
The perception APIs provide text-conditioned detection, segmentation, one-shot detection, grasp-pose sampling, and vision-language queries. These calls allow generated scripts to ground language instructions in visual observations without accessing the simulator ground truth.

Planner and collision.
The planner APIs maintain the cuRobo collision geometry from the simulated scene. Collision avoidance is enabled by default for ordinary free-space motion, and collision disabling is reserved for short, intentional contact-rich actions or known collision-model artifacts.

D.2. RoboCasa Evaluation Protocol

We evaluate each RoboCasa task in simulation using the same environment wrapper and script runtime described above. Each script generated by the coding agent is executed once per episode through the canonical evaluation harness. The harness creates the RoboCasa environment, injects the restricted Python tool namespace into the script, runs the script to completion, records videos and logs, and then queries the environment's native success function for the final score. We report binary task success as defined by RoboCasa's native success predication: an episode is counted as successful when this predicate returns true at the end of execution.

Fixed seeds and layouts.
For each reported 40-episode evaluation, we use a fixed list of seed triplets. Each triplet contains a RoboCasa random seed, a layout identifier, and a style identifier: `(seed, layout_id, style_id)`. The 40 triplets are generated once using a deterministic seed-list generator with generator seed 42, and then saved as a static evaluation file. For all reported evaluations, including smaller diagnostic subsets, we use rows from this saved file rather than regenerating a new list, so that subsets are prefixes or explicit selections from the same canonical 40-episode benchmark. The same triplet list is used for all methods compared on a task.

Fair comparison with GR00T.
When comparing generated vision scripts with GR00T, we use the same task name, same seed triplets, same initial simulator states, same camera configuration, and same RoboCasa success predicate. GR00T and the vision scripts are therefore scored on matched episodes rather than on separately sampled scenes. The task language prompt is the same RoboCasa instruction, and neither method receives oracle target poses or simulator object states during execution. The methods differ in their execution pathway: GR00T acts through its learned policy rollout, while the generated scripts compose the public perception, planning, and manipulation APIs listed in Tab. 2. We report success over the same fixed evaluation set so that differences reflect the policy or script behavior rather than different scene sampling or privileged information.

Discovered policy demonstration.
In addition to the matched-seed success rates reported in the main paper, the project website at research.nvidia.com/labs/gear/enpireincludes qualitative demonstrations of the coding-agent-discovered policy. In particular, the videos show the agent-discovered strategy of combining detection and motion planning to move the gripper to a hover pose above the target object before grasping, which is the RoboCasa strategy transferred to the real-world scissors and zip tie task in the main paper.

Result Analysis

We identified a perception bottleneck in which SAM3 can return an incorrect mask or no usable mask for small or ambiguous RoboCasa objects. Figs. 16 and 18 isolate this effect by varying image resolution and prompt wording on the same counter-to-cabinet diagnostic set. Increasing the top-camera resolution from 256×256 to 480×640 improves detection, but resolution alone does not solve all failures. Allowing the coding agent to rewrite object prompts further improves target detection and recovers several cases where the original task wording produces a distractor mask. This analysis shows that generated RoboCasa scripts are limited not only by planning and control, but also by the reliability of the perception API exposed to the agent; prompt search and resolution selection are therefore useful parts of the agent's simulation-side tool use.

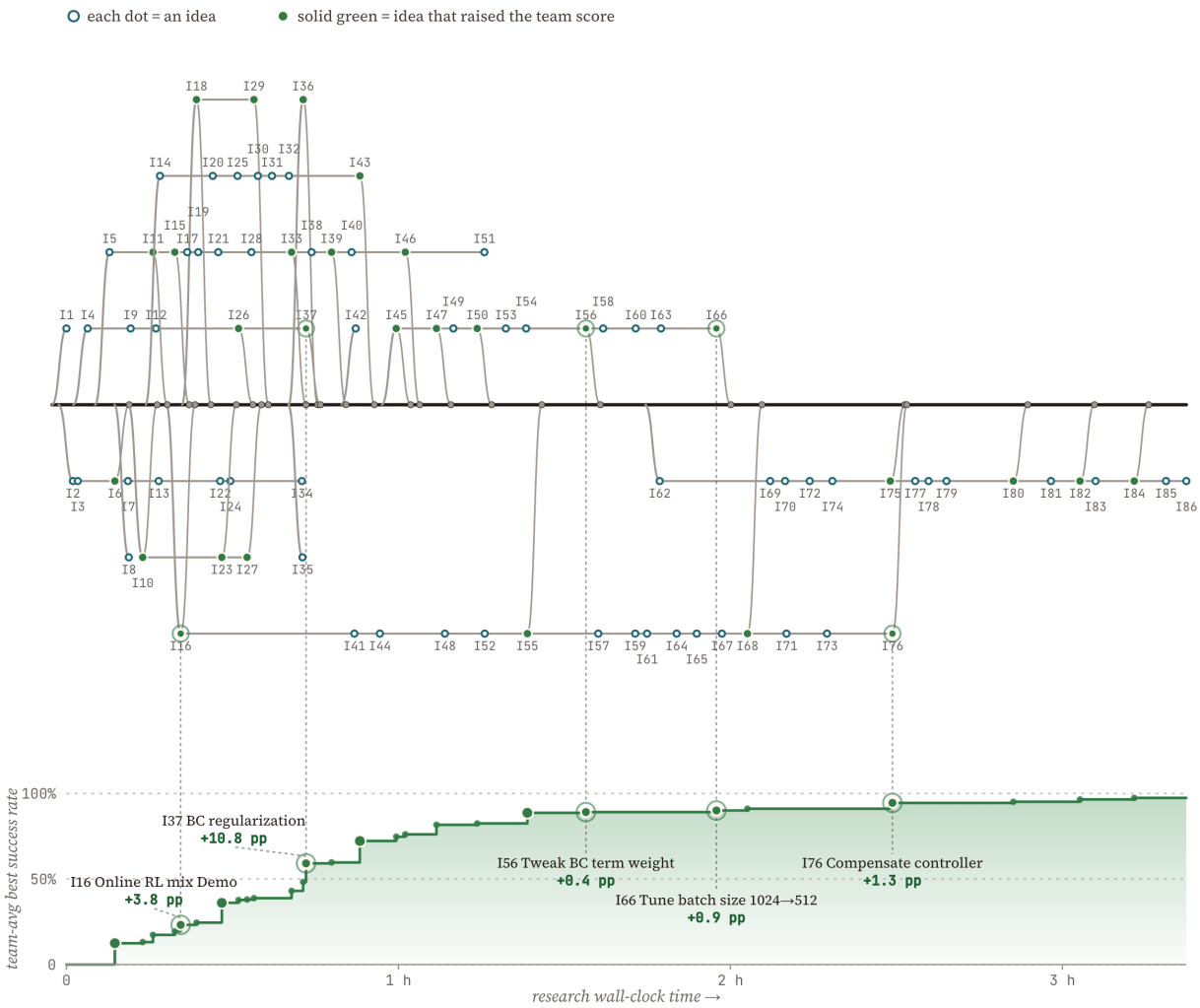


Figure 12: **Idea tree and hill-climbing progress for the pin insertion task.** *Top:* the agent's idea tree, where each node Ik is a proposed idea and each new lane is a new idea branch. Two nodes joined by a horizontal line are related ideas. Solid green nodes improved the team-average best success rate; hollow nodes were evaluated but yielded no gain. The thick black line traces the lineage of the highest-scoring idea, and circled nodes are highlighted ideas linked by dashed lines to their milestones below. *Bottom:* the team-average best success rate over research wall-clock time, sharing the time axis of the tree. Annotated milestones report each highlighted idea's improvement in percentage points (pp).

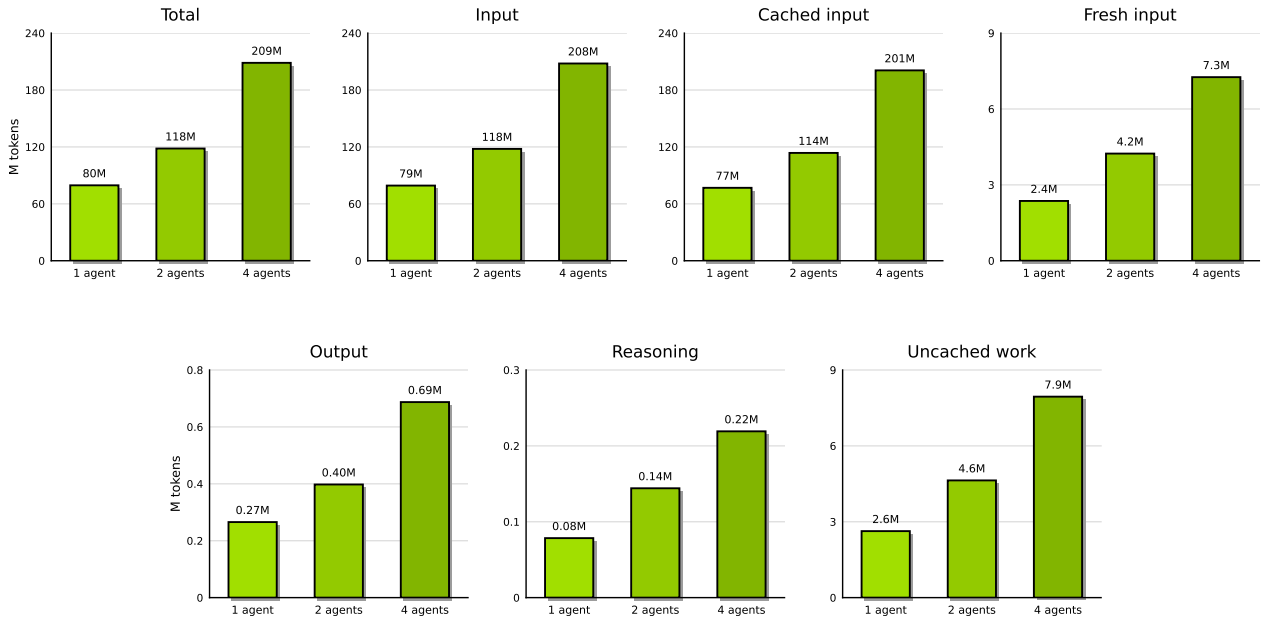


Figure 13: Token-utilization breakdown for Codex agents on the simplified real-world Push-T task. Each panel reports a different view of token consumption over the autoresearch process, separating prompt/input, generated/output, and cached-context usage where available. The figure complements the aggregate MTU analysis in the main paper by showing how token demand is distributed across planning, code editing, log inspection, and repeated policy-evaluation iterations.

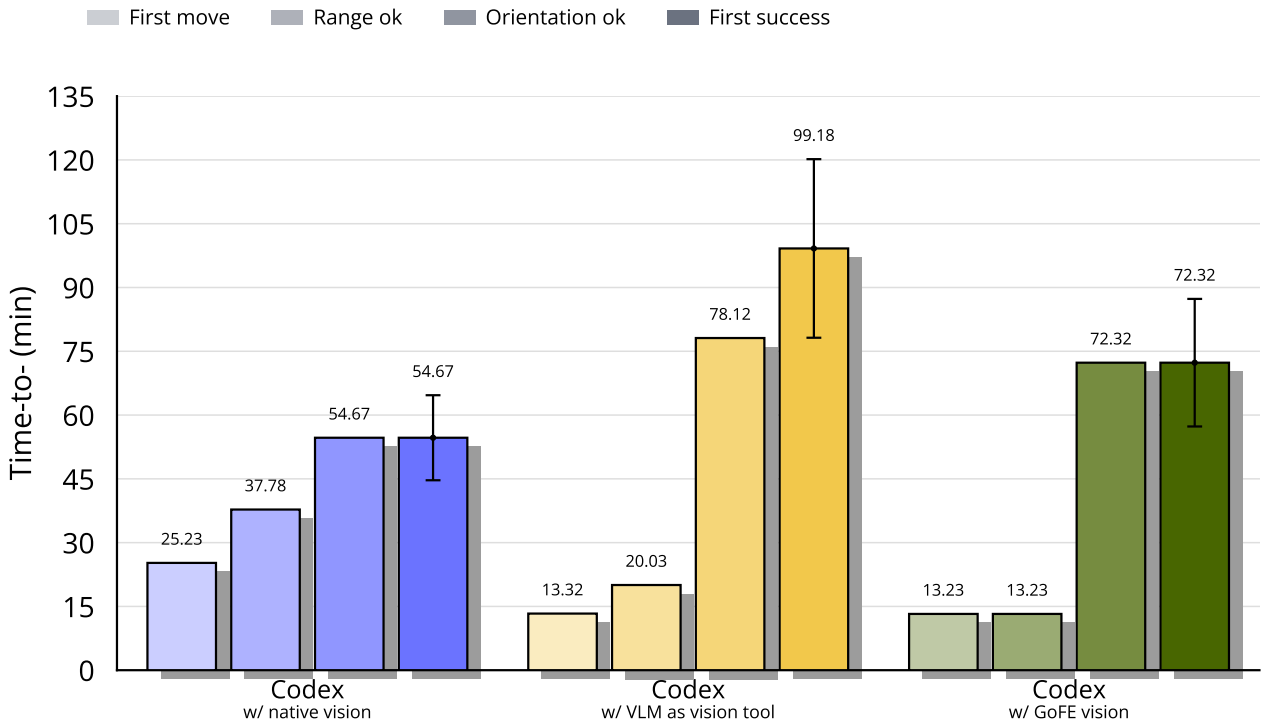


Figure 14: Stage-wise progress on the simplified real-world Push-T task under different visual-grounding conditions.

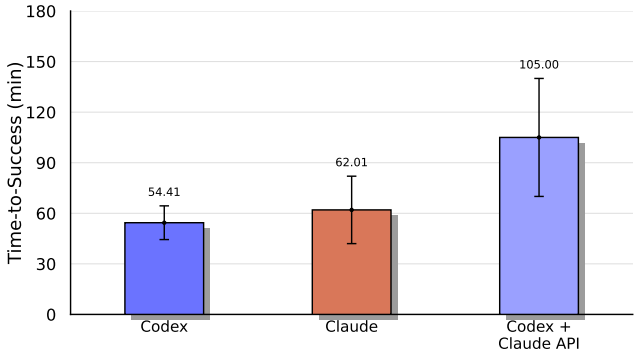


Figure 15: Time-to-success comparison across model and agent-harness configurations on the simplified real-world Push-T task. Each configuration is evaluated by the wall-clock time required to reach the predefined success criterion, capturing both model reasoning capability and the practical overhead introduced by the coding interface, tool access, execution loop, and log-inspection workflow.

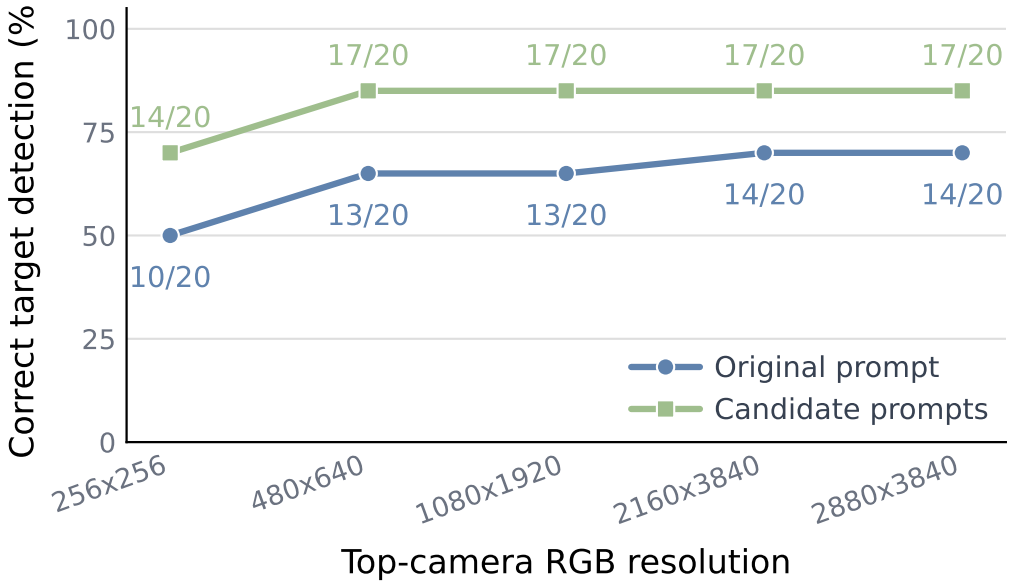


Figure 16: SAM3 target-object detection accuracy on RoboCasa counter-to-cabinet scenes as a function of top-camera RGB resolution. Each point evaluates 20 target-object queries. The original task prompt improves from 10/20 correct detections at 256×256 to 14/20 at higher resolutions, while candidate prompts generated by the coding agent improve from 14/20 to 17/20 and then plateau.

target object	256x256	480x640	1080x1920	2160x3840	2880x3840
cream cheese stick	cheese stick	stick	stick	stick	stick
water bottle	bottle	bottle	bottle	bottle	bottle
saucepan lid	wrong	wrong	lid	lid	lid
ice cube	small cube		small cube		
lemon wedge	wrong	lemon slice	wrong	wrong	wrong
garlic	wrong	small garlic			
colander	metal colander				
sugar cube	wrong	wrong	wrong	wrong	wrong
shrimp			fail	fail	
yogurt	wrong	wrong			
cookie dough ball	wrong				
marshmallow					fail
peeler					
mango					
tomato					
bread					
candle					
cupcake					
lemon					
mushroom					

Original prompt correct Candidate prompt correct Wrong mask No correct mask

Figure 17: Per-object SAM3 detection outcomes across top-camera resolutions and prompt variants for the RoboCasa counter-to-cabinet diagnostic set. Green cells indicate that the original target prompt produced a correct mask, yellow cells indicate that a candidate prompt produced a correct mask, gray cells indicate an incorrect mask, and red cells indicate that no correct mask was returned.



Figure 18: Qualitative SAM3 mask outputs for 20 RoboCasa counter-to-cabinet target objects at 480×640 resolution. Yellow boxes show selected target masks; labels indicate whether the original prompt, a candidate prompt, or a wrong mask was used.